Python Programming Assignment 1 - Source Code (Swetha N S, 1925260105A)

Repository: https://github.com/swethasundar1108/CSA0801A

Task01/task01_data.py

```
"""Task 1: data definition - inputs, named constants, types and validationrules."""
from decimal import Decimal

# Named constants (Decimal so money and percentages are exact)BASE_RATE = Decimal("30.00")        # Rs per
litre at 3.5% fat and 8.5% SNFSTD_FAT = Decimal("3.5")STD_SNF = Decimal("8.5")FAT_STEP_VALUE =
Decimal("0.40")    # Rs per litre per 0.1% fatSNF_STEP_VALUE = Decimal("0.30")    # Rs per litre per 0.1%
SNFSTEP = Decimal("0.1")MIN_FAT, MAX_FAT = Decimal("2.0"), Decimal("7.0")MIN_SNF, MAX_SNF = Decimal("6.0"),
Decimal("10.0")   # assumptionVALID_SHIFTS = ("M", "E")

# One sample entry showing the chosen data typesentry = {    "farmer_id": "F0123",        # str
(identifier, not used in arithmetic)    "litres": Decimal("8.5"),    # Decimal    "fat": Decimal("4.2"),
   # Decimal    "snf": Decimal("8.7"),       # Decimal    "shift": "M",                # str}

def is_valid(e):
    """Return True when the entry satisfies every validation rule."""    return (bool(e["farmer_id"])
     and e["litres"] >= 0            and MIN_FAT <= e["fat"] <= MAX_FAT            and MIN_SNF <= e["snf"]
<= MAX_SNF            and e["shift"] in VALID_SHIFTS)

if __name__ == "__main__":    for key, value in entry.items():        print(f"{key:10} {value!r:12}
{type(value).__name__}")    print("valid entry          :", is_valid(entry))    print("negative litres
:", is_valid({**entry, "litres": Decimal("-1")}))    print("fat 7.5 (too high)   :", is_valid({**entry,
"fat": Decimal("7.5")}))    print("shift 'X'            :", is_valid({**entry, "shift": "X"}))
```

Task02/task02.py

```
import sys
from decimal import Decimal


payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):    total = 0.0    for amount in items:        total += float(amount)    return total

def total_decimal(items):
    """Add the amounts exactly using Decimal built from the strings."""    total = Decimal("0")    for
amount in items:        total += Decimal(amount)    return total

def total_decimal_from_float(items):
    """Variant: Decimal(float(amount)) - inherits binary float error."""    total = Decimal("0")    for
amount in items:        total += Decimal(float(amount))    return total

def total_paise(items):
    """Integer paise: exact, fastest, but needs conversion in and out."""    paise = 0    for amount in
items:        paise += round(float(amount) * 100)    return paise / 100, paise

def total_float_rounded(items):
```

```
    total = 0.0
    for amount in items:
        total = round(total + float(amount), 2)
    return total

print("float :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3, Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
print("Decimal(float(amount)):", total_decimal_from_float(payments))
print("integer paise:", total_paise(payments))
print("float + round each add:", total_float_rounded(payments))
print("getsizeof float :", sys.getsizeof(1.5))
print("getsizeof int    :", sys.getsizeof(3340))
print("getsizeof Decimal:", sys.getsizeof(Decimal("33.40")))
for v in (10, 10.0, "10", True, [10, 8.5]):
    print(repr(v), type(v))
print(7 / 2, 7 // 2, 7 % 2, -7 // 2)
```

Task03/constants_demo.py

```
BASE_RATE = 30.00
STD_FAT = 3.5
STD_SNF = 8.5
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
STEP_SIZE = 0.1
MIN_FAT, MAX_FAT = 2.0, 7.0
RATE_CARD = {"base": 30.0, "fat": 0.40, "snf": 0.30}

x = 10; print(type(x)); x = "ten"; print(type(x))   # dynamic typing
a, b = 3, 5; a, b = b, a; print(a, b)               # swap
c = d = 0; e, f = 1, 2; print(c, d, e, f)           # multiple assignment
BASE_RATE = 99.0   # Python allows this: constants are convention only
print(BASE_RATE)
```

Task04/expressions.py

```
import math
print("(4.2-3.5)/0.1 =", (4.2 - 3.5) / 0.1, "-> round:", round((4.2 - 3.5) / 0.1))
print("(3.1-3.5)/0.1 =", (3.1 - 3.5) / 0.1)
print("(8.7-8.5)/0.1 =", (8.7 - 8.5) / 0.1)
print("-2**2 =", -2 ** 2, " (-2)**2 =", (-2) ** 2, " 2**3**2 =", 2 ** 3 ** 2)
print("negatives: round(-2.5)=", round(-2.5), "int(-2.5)=", int(-2.5), "floor(-2.5)=", math.floor(-2.5))
BASE, FATV, SNFV = 30.0, 0.40, 0.30
for fat, snf in [(4.2, 8.5), (3.1, 8.5)]:


    raw = (fat - 3.5) / 0.1
    for name, fn in [("round", round), ("int", int), ("floor", math.floor)]:
        steps = fn(raw)
        print(f"fat {fat}: raw={raw!r:20} {name:5} steps={steps:3} rate={BASE + steps*FATV:.2f}")
fat = 4.2
print(2.0 <= fat <= 7.0, 2.0 <= fat and fat <= 7.0, not (fat < 2.0 or fat > 7.0))
```

Task05/input_demo.py

```
def parse_float(text):


    try:
        return float(text)
    except ValueError:
        return None

def parse_with_string_checks(text):
    t = text.strip()
    if t.replace(".", "", 1).isdigit():
        return float(t)
    return None

for s in ["8.5", "8,5", " 8.5 ", "eight"]:
    print(f"{s!r:8} try/except -> {parse_float(s)!s:5} | string checks -> {parse_with_string_checks(s)}")
amount = 283.9
print(f"[{amount:8.2f}] [{'F001':>6}] [{12:5d}]")

def read_float(prompt):          # interactive version
    while True:
        value = parse_float(input(prompt))
        if value is not None:
            return value
        print("Invalid number, try again.")
```

Task06/functions.py

```
BASE_RATE, FAT_STEP_VALUE, SNF_STEP_VALUE = 30.00, 0.40, 0.30

def rate_per_litre(fat, snf):
    """Return price per litre for the given fat and SNF percentage."""
```

```
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)

def payout(litres, fat, snf):
    """Return payment amount for one delivery."""
    return round(litres * rate_per_litre(fat, snf), 2)

def ten_day_total(entries):
    """Return total payment for a farmer; entries = [(litres, fat, snf), ...]."""
    return round(sum(payout(l, f, s) for l, f, s in entries), 2)

if __name__ == "__main__":
    week = [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]
    print(ten_day_total(week))
    square = lambda x: x * x
    print(square(4))
```

Task07/milkrate.py

```
"""Rate card of the village milk co-operative."""

BASE_RATE   =   30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30

def rate_per_litre(fat, snf):
    """Return the price per litre, rounded to two decimals."""
    fat_steps = round((fat - 3.5) / 0.1)
    snf_steps = round((snf - 8.5) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE, 2)

def payout(litres, fat, snf):
    """Return the payment amount rounded to two decimals."""
    return round(litres * rate_per_litre(fat, snf), 2)

if __name__ == "__main__":
    for litres, fat, snf in [(10, 3.5, 8.5), (8.5, 4.2, 8.7), (12, 3.1, 8.3)]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
        print(f"{litres:5} L fat {fat} snf {snf}" f" rate {rate:6.2f} pay {pay:8.2f}")
```

Task07/use_milkrate.py

```
import sys
import milkrate as mr
print("Payout for 20 L, 4.0 fat, 8.6 SNF:", mr.payout(20, 4.0, 8.6))
print("milkrate cached in sys.modules:", "milkrate" in sys.modules)
```

Task09/milk_calc.py

```
"""Final calculator: Decimal arithmetic, validation, functions."""
from decimal import Decimal, ROUND_HALF_UP



BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STD_FAT, STD_SNF, STEP = Decimal("3.5"), Decimal("8.5"), Decimal("0.1")
MIN_FAT, MAX_FAT = Decimal("2.0"), Decimal("7.0")
MIN_SNF, MAX_SNF = Decimal("6.0"), Decimal("10.0")   # assumption
PAISE = Decimal("0.01")

def money(x):
    return x.quantize(PAISE, rounding=ROUND_HALF_UP)

def parse_decimal(text):
    """Convert text to Decimal; raise ValueError for bad input (e.g. '8,5', 'eight')."""
    try:
        value = Decimal(text.strip())
    except Exception:
        raise ValueError(f"not a number: {text!r}")
    if not value.is_finite():
        raise ValueError(f"not a finite number: {text!r}")
    return value

def validate(litres, fat, snf):
    if litres < 0:
        raise ValueError("litres must be non-negative")
    if not MIN_FAT <= fat <= MAX_FAT:
        raise ValueError("fat must be between 2.0 and 7.0")
    if not MIN_SNF <= snf <= MAX_SNF:
        raise ValueError("SNF must be between 6.0 and 10.0")

def steps(value, standard):
```

```
    return ((value - standard) / STEP).to_integral_value(rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf):
    return money(BASE_RATE + steps(fat, STD_FAT) * FAT_STEP_VALUE
                 + steps(snf, STD_SNF) * SNF_STEP_VALUE)

def payout(litres, fat, snf):
    return money(litres * rate_per_litre(fat, snf))

def ten_day_total(entries):
    return sum((payout(l, f, s) for l, f, s in entries), Decimal("0.00"))

def process(litres_t, fat_t, snf_t):
    l, f, s = parse_decimal(litres_t), parse_decimal(fat_t), parse_decimal(snf_t)
    validate(l, f, s)
    return payout(l, f, s)
```

Task09/test_calculator.py

```
from decimal import Decimal as D
from milk_calc import *

cases = [
 ("above standard", ("8.5","4.2","8.7"), D("283.90")),
 ("below standard", ("12","3.1","8.3"), D("333.60")),
("boundary 3.5/8.5", ("10","3.5","8.5"), D("300.00")),
("lowest fat 2.0", ("10","2.0","8.5"), D("240.00")),
("highest fat 7.0", ("10","7.0","8.5"), D("440.00")),
("invalid text", ("10","eight","8.5"), ValueError),
 ("comma decimal", ("10","3,5","8.5"), ValueError),
 ("negative litres", ("-5","3.5","8.5"), ValueError),
 ("fat too high", ("10","7.1","8.5"), ValueError),
 ("fat too low", ("10","1.9","8.5"), ValueError),
]
print(f"{'Case':18}{'Input':26}{'Expected':>12}{'Actual':>12}  Result")
for name, inp, exp in cases:
    try:
        act = process(*inp)
    except ValueError:
        act = ValueError
    ok = act == exp
    exp_s = getattr(exp, "__name__", str(exp))
    act_s = getattr(act, "__name__", str(act))
    status = "PASS" if ok else "FAIL" print(f"{name:18}{str(inp):26}
    {exp_s:>12}{act_s:>12} {status}") assert ok


rows = [("10", "3.5", "8.5"), ("8.5", "4.2", "8.7"), ("12", "3.1", "8.3"),
("20", "4.0", "8.6"), ("5", "3.6", "8.4")] entries
= [(D(a), D(b), D(c)) for a, b, c in rows] manual =
(D("300.00") + D("283.90") + D("333.60")
         + D("20") * D("32.30") + D("5") * D("30.10"))
print("5-entry total actual:", ten_day_total(entries), "manual:", manual)
assert ten_day_total(entries) == manual
print("ALL TESTS PASSED")
```

Task10/experiments.py

```
import random, time
from decimal import Decimal, ROUND_HALF_UP
random.seed(1)
P = Decimal("0.01")


def gen(n):  # exact payments in 3 decimals, as products litres * rate can give
    return [Decimal(random.randint(10000, 60000)) / 1000 for _ in range(n)]
print(f"{'n':>8}{'float total':>22}{'Decimal total':>16}{'round each':>14}{'t_float(s)':>12}{'t_dec(s)':>10}")
for n in (300, 6000, 150000):
    data = gen(n); fl = [float(x) for x in data]
    t = time.perf_counter(); tf = sum(fl); t_f = time.perf_counter() - t
    t = time.perf_counter(); td = sum(data, Decimal(0)); t_d = time.perf_counter() - t
    each = sum((x.quantize(P, rounding=ROUND_HALF_UP) for x in data), Decimal(0))
    print(f"{n:>8}{tf:>22.10f}{td.quantize(P, rounding=ROUND_HALF_UP):>16}{each:>14}{t_f:>12.6f}{t_d:>10.6f}")
# rate-table vs recalculation
import sys; sys.path.insert(0, "../Task09")
from milk_calc import rate_per_litre
def random_pair():
    fat = Decimal(str(round(random.uniform(2, 7), 1)))
    snf = Decimal(str(round(random.uniform(7, 9.5), 1)))
    return fat, snf
pairs = [random_pair() for _ in range(150000)]
t = time.perf_counter(); [rate_per_litre(f, s) for f, s in pairs]; t_calc = time.perf_counter() - t
table = {}
t = time.perf_counter()
for p in pairs:
    if p not in table: table[p] = rate_per_litre(*p)
    _ = table[p]
```

```
t_tab = time.perf_counter() - t
print(f"recalculate 150000: {t_calc:.3f}s | dict lookup: {t_tab:.3f}s | distinct pairs k = {len(table)}")
```

Task12/original_vs_new.py

```
"""Worked examples: original (Task 7) design applied to two-decimal analyser readings."""
import sys
sys.path.insert(0, "../Task07")
import milkrate as old
import seasonal as new
from decimal import Decimal as D



print(f"{'fat':>5}{'snf':>6}{'raw fat steps':>22}{'old steps':>10}{'old rate':>10}{'new rate (normal)':>19}")
for fat, snf in [(4.25, 8.5), (4.35, 8.5), (4.45, 8.5), (4.15, 8.5), (3.55, 8.65)]:
    raw = (fat - 3.5) / 0.1
    print(f"{fat:>5}{snf:>6}{raw!r:>22}{round(raw):>10}{old.rate_per_litre(fat, snf):>10.2f}"
          f"{new.money(new.rate_per_litre(D(str(fat)), D(str(snf)))):>19}")
print("summer, original design cannot express: base 32.00 / fat step 0.45 / bonus")
```

Task12/seasonal.py

```
"""Seasonal rate card + bonus. Fat/SNF readings may have 2 decimals; steps are pro-rata (fractional)."""
from decimal import Decimal as D, ROUND_HALF_UP
PAISE = D("0.01")
STD_FAT, STD_SNF, STEP = D("3.5"), D("8.5"), D("0.1")
SNF_STEP_VALUE = D("0.30")
RATE_CARDS = {


    "normal": {"base": D("30.00"), "fat_step": D("0.40")},
    "summer": {"base": D("32.00"), "fat_step": D("0.45")},   # April-June
}
BONUS_LIMIT, BONUS_PER_LITRE = D("500"), D("0.50")
SUMMER_MONTHS = (4, 5, 6)

def money(x): return x.quantize(PAISE, rounding=ROUND_HALF_UP)
def season_for_month(month): return "summer" if month in SUMMER_MONTHS else "normal"

def rate_per_litre(fat, snf, season="normal"):
    card = RATE_CARDS[season]
    return card["base"] + (fat - STD_FAT) / STEP * card["fat_step"] + (snf - STD_SNF) / STEP * SNF_STEP_VALUE

def payout(litres, fat, snf, season="normal"):
    return litres * rate_per_litre(fat, snf, season)

def bonus(total_litres):
    return total_litres * BONUS_PER_LITRE if total_litres > BONUS_LIMIT else D("0")

def ten_day_total(entries, season="normal"):
    total = sum((payout(l, f, s, season) for l, f, s in entries), D("0"))
    litres = sum((l for l, _, _ in entries), D("0"))
    return money(total + bonus(litres))

if __name__ == "__main__":
    print("rate 4.25/8.65 normal:", money(rate_per_litre(D("4.25"), D("8.65"))))
    print("rate 4.25/8.65 summer:", money(rate_per_litre(D("4.25"), D("8.65"), "summer")))
    small = [(D("10"), D("3.50"), D("8.50"))] * 10                    # 100 L
    big = [(D("60"), D("4.20"), D("8.70"))] * 10                      # 600 L
    assert ten_day_total(small, "normal") == D("3000.00")
    assert ten_day_total(small, "summer") == D("3200.00")             # 100 x 32
    assert ten_day_total(big, "normal") == money(600 * D("33.40") + 300)
    assert ten_day_total(big, "summer") == money(600 * (D("32.00") + 7*D("0.45") + 2*D("0.30")) + 300)
    assert bonus(D("500")) == 0 and bonus(D("500.01")) > 0            # strictly more than 500
    print("small normal/summer:", ten_day_total(small), ten_day_total(small, "summer"))
    print("big normal/summer :", ten_day_total(big), ten_day_total(big, "summer"))
    print("ALL SEASONAL TESTS PASSED")
```